

Business Trips - Application

Aspects of a Business Trips App, Backend

Beschreibung

In diesem Dokument geht es um die Umsetzung von Konzepten für eine Business Trip App. Die Technologien, welche eingeführt werden, sind weiter unten beschrieben.

Folgende Anforderungen muss die App erfüllen:

- Der Projektmanager weist mehrere Business Trips den Mitarbeitern zu (sieht alle BT, kann Zuordnung bearbeiten)
- Mitarbeiter bearbeitet seine BT
 - Bearbeitet seine BT, Daten, Flüge Status
- Projektmanager hat Kostenübersicht über alle Spesen
- Mitarbeiter führt seine Spesenliste
- Architektur: Trennung der View von der Logik React
 - Backend: Java Spring Boot und Spring Data JPA / REST

Frontend: Web App mit

React Zeitbedarf:

Hilfsmittel:

Methode/Sozialform:

Kompetenzen:

Abgaben

12 Lektionen

Java, Spring Boot, React

PA  (Backend, Frontend mit React)

-

1.  Frontend React

2.  Backend Java Spring Boot/Spring REST

Legende:



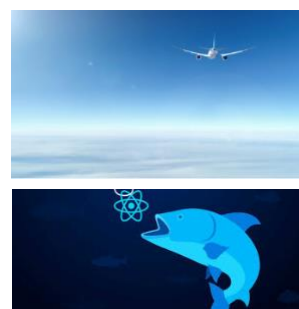
Partnerarbeit



Dokumentation



Code



Inhaltsverzeichnis

1	START.....	3
1.1	 ENTITYS FÜR BUSINESS-TRIP	3
1.1.1	Beispiel Datenmodell.....	3
1.1.2	 Aufgabe Business-Trip-Entity.....	4
1.2	 REPOSITORY FÜR BUSINESSTRIP.....	5
1.2.1	 Aufgabe BusinessTripRepository	6
1.3	 TEST-DATEN	6
1.3.1	 Aufgabe Testdaten	7
1.4	 AUFGABE – PRÜFT DIE GANZE BUSINESS-TRIP APP	7
1.5	 AUFGABE – FÜHRT DIE TESTS MIT POSTMAN DURCH.....	7

1 Start

Übernahme die letzte Version der biztrips React App mit Spring Web/ React
Folgenden Stack werden wir benutzen:

1. Java, Maven als den Build Manager
2. Spring Boot mit Spring Web / Restful

Die Aufgabe verwendet dabei folgende Konzepte kennen lernen:

- Spring Web – RestController / View React
- Datenobjekte / Listen
- Zweischichtige Architektur / Repositories von Spring
- Dependency Injection mit @Autowired
- JPA Entities

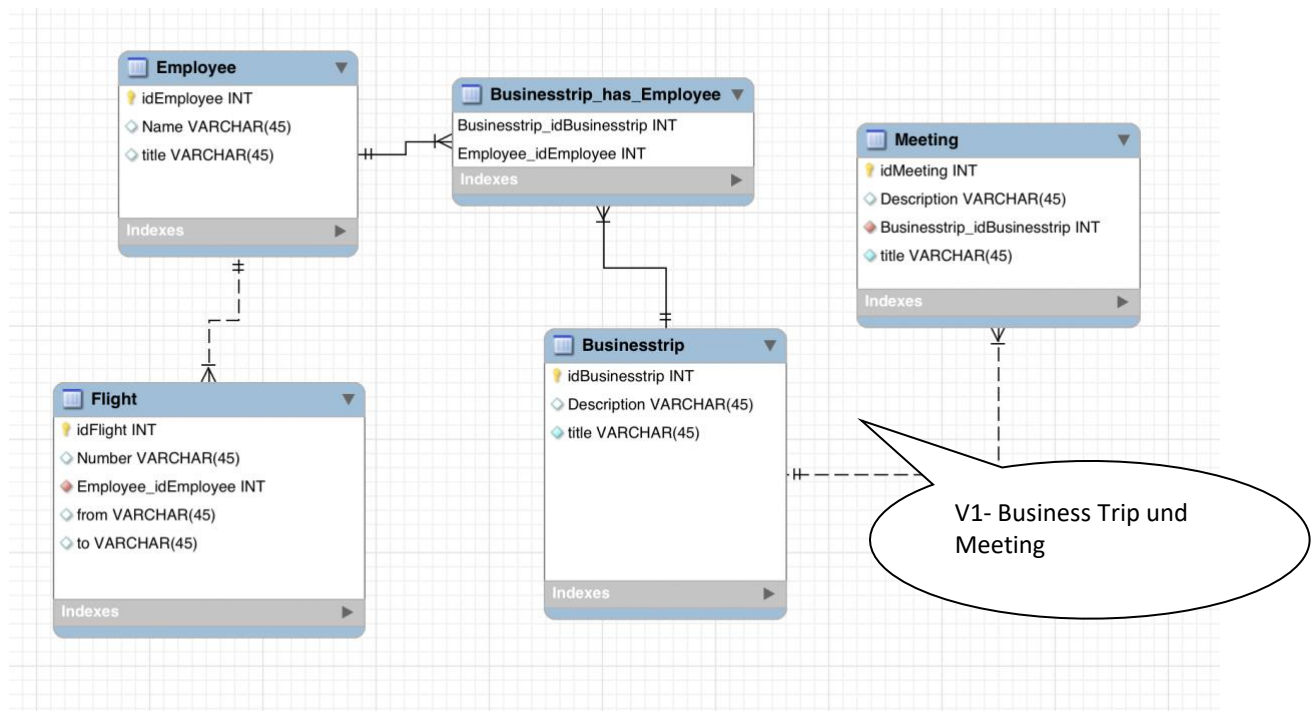
1.1 Entitäts für Business-Trip

Für unsere Entitäts, also die fachlichen Klassen unserer App, brauchen wir Spring Data JPA, hier ein Beispiel:
<https://spring.io/guides/gs/accessing-data-jpa/> dies erleichtert uns das Model in die Datenbank zu speichern!

1.1.1 Beispiel Datenmodell

Von der obersten Schicht, der View gehen wir nun zuunterst zur Datenbank, *kopfüber!*

Überlegen wir uns kurz, anhand des Datenmodell – Entwurfs welche Komponenten wir brauchen! Wir werden hier einfach starten und uns zunächst um die zwei Entitäten Businessstrip und Meeting kümmern! Den Rest machen wir dann später!



<https://spring.io/guides/gs/accessing-data-jpa/>

Das Modell: hier am Beispiel einer BusinessTrip - Entity:

```
@Entity
public class BusinessTrip implements Serializable {

    private static final long serialVersionUID = 67027563808382509L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;
    private LocalDateTime startTrip;
    private LocalDateTime endTrip;

    @OneToMany(mappedBy = "businessTrip")
    private List<Meeting> meetings;

    // constructors / getter and setter omitted
}
```

Indicates that the persistence provider must assign primary keys for the entity using a database identity column.

Beziehungen im Datenmodell definiert

```
@Entity
public class Meeting implements Serializable {

    private static final long serialVersionUID = 6705527563808382509L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;

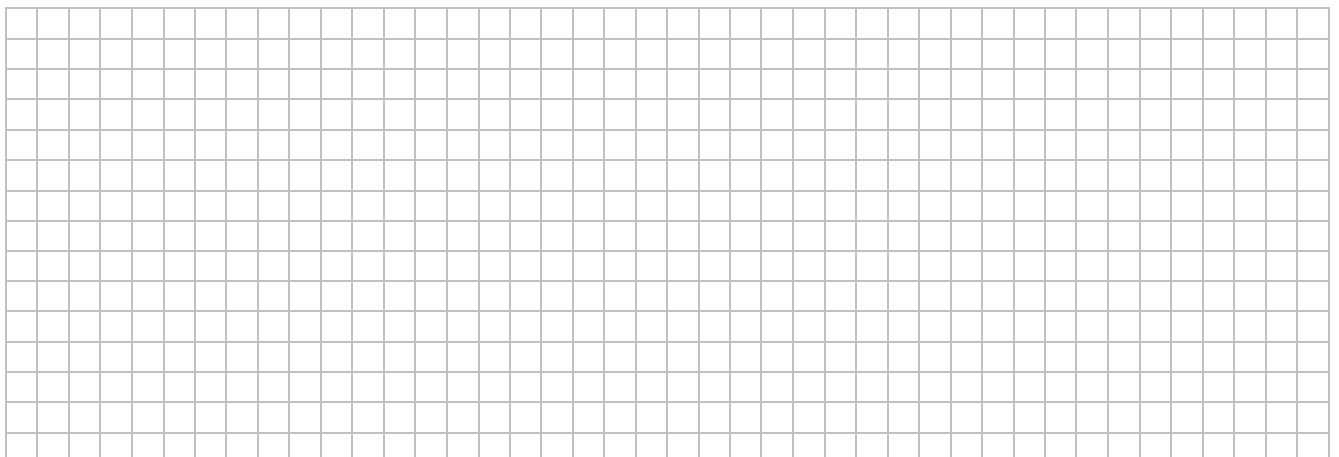
    @ManyToOne
    @JoinColumn(name = "business_trip_ids")
    private BusinessTrip businessTrip;
```

Beziehungen im Datenmodell definiert (Beidseitig)



1.1.2 Aufgabe Business-Trip-Entity

- a) Mach euch mit dem code vertraut, code lesen und verstehen können!



1.2 Repository für BusinessTrip

Bei den Repositories oder im Java Jargon DAO's, also die Datenbank – nahe Schicht, nimmt uns Spring sehr viel Arbeit ab, denn die Klassen CrudRepository oder JpaRepository sind magically schon vorbereitet siehe <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#repositories>

4.1. Core concepts

The central interface in the Spring Data repository abstraction is `Repository`. It takes the domain class to manage as well as the ID type of the domain class as type arguments. This interface acts primarily as a marker interface to capture the types to work with and to help you to discover interfaces that extend this one. The `CrudRepository` provides sophisticated CRUD functionality for the entity class that is being managed.

Example 3. CrudRepository interface

```
public interface CrudRepository<T, ID extends Serializable>
    extends Repository<T, ID> {

    <S extends T> S save(S entity);           ❶

    Optional<T> findById(ID primaryKey);     ❷

    Iterable<T> findAll();                   ❸

    long count();                             ❹

    void delete(T entity);                   ❺

    boolean existsById(ID primaryKey);       ❻

    // ... more functionality omitted.
}
```

- ❶ Saves the given entity.
- ❷ Returns the entity identified by the given ID.
- ❸ Returns all entities.
- ❹ Returns the number of entities.
- ❺ Deletes the given entity.
- ❻ Indicates whether an entity with the given ID exists.

Dank den neuen Default Methoden in Interfaces, ist das Interface CrudRepository schon vorbereitet mit eben den CRUD-Methoden (Create, Read, Update und Delete), wie praktisch, dass man sich nicht mehr kümmern muss und die Methoden schon magisch da sind! Die Methoden welche zusätzlich verwendet werden können wir im interface hinzufügen, siehe das Beispiel vom CustomerRepository:

```
public interface CustomerRepository extends CrudRepository<Customer, Long> {

    List<Customer> findByLastName(String lastName);

}
```

Hier fügen wir die zusätzlichen Methoden welche wir brauchen, wir halten uns an die Namenskonvention und Spring erstellt uns magisch die SQL-Abfragen im Hintergrund, wow!

CrudRepository Interface

CrudRepository is an interface and extends Spring data **Repository** interface. **CrudRepository** provides generic CRUD operation on a repository for a specific type. It has generic methods for CRUD operation. To use **CrudRepository** we have to create our interface and extend **CrudRepository**. We need not to implement our interface, **its implementation will be created automatically at runtime**. Find some of **CrudRepository** methods.

1.2.1 Aufgabe BusinessTripRepository

- a) Code lesen und verstehen: BusinessTripRepository ähnlich dem CustomerRepository Beispiel!

[illegible]

1.3 Test-Daten

Um Testdaten einzufügen kann im Spring – Ecosystem ein CommandLineRunner-Bean-Methode erstellt werden welches beim Starten aufgerufen wird! Dies kann direkt in der Application Class erfolgen oder in einer separaten Initializer Class.

Für weitere Infos <https://spring.io/guides/gs/accessing-data-jpa/>

Bean in der Application Class:

```
@Bean
public CommandLineRunner demo(CustomerRepository repository) {
    return (args) -> {
        // save a couple of BusinessTrips
        BusinessTrip bt01 = new BusinessTrip(1L, "BT01", "San Francisco World Trade Center on new Server/IOT/Client ", LocalDateTime.of(2021, 2, 13, 9, 00),
LocalDateTime.of(2021, 2, 15, 16, 56));

        BusinessTrip bt02 = new BusinessTrip(2L, "BT02", "Santa Clara Halley on new Server/IOT/Client", LocalDateTime.of(2021, 6, 23, 9, 00),
LocalDateTime.of(2021, 6, 27, 16, 56));

        BusinessTrip bt03 = new BusinessTrip(3L, "BT03", "San Cose City Halley on Docker/IOT/Client", LocalDateTime.of(2021, 12, 13, 9, 00),
LocalDateTime.of(2021, 12, 15, 16, 56));

        businessTripRepository.save(bt01);
        businessTripRepository.save(bt02);
        businessTripRepository.save(bt03);

        // save a couple of meetings

        Meeting one = new Meeting(1L, "One Conference", "Keynote on One Conference", bt01);
        Meeting zero = new Meeting(2L, "Zero Conference", "Workshop Zero on One Conference", bt01);
        Meeting handsOn = new Meeting(3L, "One Conference", "HandsOn on One Conference", bt02);

        Meeting workOn = new Meeting(4L, "One Conference", "Keynote on One Conference", bt02);
        Meeting stayTuned = new Meeting(5L, "One Conference", "Keynote on One Conference", bt03);
    }
}
```

}

Entity)



b) Beschreibung der Testdaten in der Dokumentation

[illegible]

- a) Prüft die Entitäts und deren Beziehungen kritisch
- b) Prüft die Package Struktur, ändert Sie je nach Anforderungen
- c) bereinigt den code und verwendet Spring Boot v3 :

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.6</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
```


- d) Prüft alle Endpoints mit Postman

Softwareentwicklung

Business Trip Project

